

1 从应用到镜像——应用管理的起点

概述

亮点

在机器人应用部署过程中，真正困难的往往不是复制代码，而是重新构建一整套能让代码运行起来的系统环境。

以部署一个导航系统为例，当开发者拿到一个主机后，首先需要配置支持导航框架的操作系统，以及运行的软件，最后才是拉取所需要的导航系统仓库，**整个过程费事费力且非常容易出错**，为此我们一直在思考能否找到一个封装形式，将这一整条部署链路上的操作系统等部分封装成一个整体，并提供相应的管理系统，实现对机器人应用的管理，从而提高开发者部署机器人应用时的效率？

为了解决这个问题，本系统提出了一个核心概念：**机器人镜像 Robot Image**。

它不再只管理应用代码，而是把操作系统、依赖、中间件、开发环境、工具链和应用代码统一封装成一个完整的软件栈。这样，机器人应用就可以以“镜像”的方式被保存、迁移、克隆和部署，实现接近“开箱即用”的效果。

基于这样一个核心概念，本系统的亮点主要体现在三个方面。

第一，用机器人镜像统一封装完整软件栈。

系统管理的不再是单一代码仓库，而是把“应用运行所需的一切”统一看作一个完整的软件栈进行管理。

第二，将机器人镜像中心化存储到服务器侧。

镜像不再分散在不同机器人主机上，而是集中托管在服务器的虚拟化平台中，方便后续进行模板化、克隆、恢复和统一运维。

第三，为自动部署 pipeline 和智能分发调度打基础。

当镜像能够被统一注册、统一查询、统一模板化和统一克隆之后，系统就可以进一步发展出自动部署和智能资源调度能力。

从而解决掉：

1. 在机器人应用部署过程中，重复配置完整环境耗时耗力的问题
2. 由于机器人应用运维的各个环节高度依赖人工手动执行，导致容易引入环境bug的问题。

所解决的问题

1. 在机器人应用部署过程中，重复配置完整环境耗时耗力的问题
2. 由于机器人应用运维的各个环节高度依赖人工手动执行，导致容易引入环境bug的问题

为什么镜像能解决这个问题

前面我们提到，机器人应用部署的核心痛点，是每次都要重复配置操作系统、依赖库、中间件、驱动程序、开发工具和应用代码。

那么，为什么“镜像”能够解决这个问题？

首先，我们需要明确什么是机器人镜像。

机器人镜像并不是简单地把应用代码打成一个压缩包，也不是只保留一个运行时容器。它更像是一份完整的系统快照，把应用代码、开发工具、运行环境、依赖、中间件、操作系统以及底层配置这些“能让机器人应用运行和开发所需的一切”统一表示出来。换句话说，机器人镜像管理的对象不再是某一个应用，而是一整套完整的软件栈。PPT 中也将机器人镜像定义为对底层系统、运行环境、开发工具和应用本体的统一表示，并强调它超越了单纯的代码打包。

正因为机器人镜像本质上是一份完整的磁盘镜像，所以它天然具有很强的可迁移性。

作为磁盘镜像，它并不绑定在某一台固定主机上。任何带有存储能力的介质，例如移动硬盘、NUC、个人电脑、服务器，或者车载主机，都可以成为机器人镜像的载体。借助 Clonezilla 这样的磁盘克隆工具，镜像可以在不同载体之间迁移。例如，我们可以把车载主机上的机器人系统克隆到移动硬盘中，再让个人电脑从这块移动硬盘启动，从而获得一个与车载主机几乎完全一致的机器人系统环境。也就是说机器人镜像可以在移动硬盘、NUC、服务器和车载主机之间迁移，从而实现“**环境随身走**”。

演示：利用 CLONEZILLA 实现镜像快速迁移



这种可迁移性带来的核心价值，是维护机器人应用在“开发—调试—测试—部署”全生命周期中的环境一致性。

机器人开发和普通软件开发不太一样。很多时候，我们不仅要运行应用，还要在同一个系统环境中继续开发代码、调试传感器、验证驱动、检查通信、接入硬件。如果每一次测试都需要重新封装、重新部署、重新配置环境，那么开发成本会非常高，而且每一次人工配置都可能引入新的环境差异。

而机器人镜像保留的是一整套可以直接进入开发和测试的系统环境。因此，开发、调试、测试和部署不再是四个彼此割裂的环节，而是同一个镜像在不同载体之间的流转。开发时使用这份镜像，调试时仍然使用这份镜像，测试和最终部署到机器人主机时也使用同一份镜像。这样就能最大程度避免“开发机能运行，车载机不能运行”的经典环境差异问题。

为什么要针对镜像设置中心化存储

但是，仅仅有机器人镜像还不够。

如果镜像仍然分散在各个机器人主机、移动硬盘或者个人设备上，那么每次部署、备份、迁移或者恢复时，仍然需要人工去查找镜像、确认版本、选择目标设备并执行克隆。这样虽然解决了一部分环境一致性问题，但管理效率仍然不高。

所以，在机器人镜像具有可迁移性的基础上，系统进一步提出：**把机器人镜像集中存储到服务器上，并由服务器进行统一管理。**

这样一来，服务器就不只是一个普通的文件存储位置，而是变成了统一的机器人镜像仓库。所有经过验证的机器人镜像都可以集中存放在服务器侧，后续无论是模板化、克隆、恢复、维护，还是自动部署和智能分发，都可以从这个统一入口开始。PPT 第 7 页也指出，如果镜像仍然分散在各个机器人主机上，管理仍然低效；中心化存储则为自动克隆、恢复、维护和统一运维提供了物理前提。

基于中心化存储的各项功能实现

在工程实现上，本系统进一步使用虚拟机作为机器人镜像在服务器端的物理载体，并通过 PVE 虚拟化平台对这些镜像进行统一控制。这里需要强调的是，虚拟机并不是机器人镜像唯一的运行形式，它只是服务器侧便于存储和管理的一种载体。由于机器人镜像本身具有可迁移性，它仍然可以被克隆到机器人裸机上运行。PVE 的作用，是让服务器能够程序化地完成镜像存储、模板化、克隆和运维，从而把“人工查找和复制镜像”转变为“系统统一管理和复制镜像”。

镜像形式化

在物理实现了镜像的统一存储之后，仍需管理系统对镜像进行智能运维，此时，镜像就不能只是一个概念，还必须变成系统里可被查询、调度、更新的对象。所以我们在系统中对机器人镜像进行了形式化，把它定义成一个统一的数据对象：RobotAppImage。可以把它理解成每张镜像的一张“登记卡”。这张登记卡会记录镜像是谁、叫什么、哪个版本、来自哪台 VM、当前是什么状态、是不是模板。有了这种统一定义之后，镜像才能真正进入后续的自动管理系统。

代码块

```
1 class RobotAppImage:
2     """
3     机器人应用镜像实体
```

```

4
5     代表一个包含了OS、运行环境、开发工具和应用本体的虚拟机镜像。
6     """
7     def __init__(
8         self,
9         image_id: str,
10        name: str,
11        version: str,
12        source_vm_id: int,
13        *,
14        is_template: bool = False,
15    ):
16        self.image_id = image_id
17        self.name = name
18        self.version = version
19        self.source_vm_id = source_vm_id # 在PVE源服务器上的VM ID模板
20        self.status = "stopped"         # running, stopped, deploying
21        self.is_template = bool(is_template)
22
23    def __repr__(self):
24        return f"<RobotApp: {self.name} (v{self.version})>"

```

主机与镜像资源注册

系统里的“镜像对象”，怎么和服务器的实际虚拟机对应起来？使用RobotApplImage对象里的source_vm_id作为桥梁，可以将系统里的“镜像对象”绑定到服务器上的对应虚拟机。具体而言，系统在注册阶段会基于服务器注册数据中心，之后读取服务器中的VM列表，然后自动为这些VM创建对应的RobotApplImage对象。最后将这些对象纳入管理系统，为后续自动化流水线提供Handle。这样，系统就知道哪一个逻辑上的机器人镜像，对应的是哪一台实际的虚拟机。具体实现代码如下：

代码块

```

1     def _step_2_discover_and_register(self) -> None:
2         """联动式资源注册：注册节点 -> 询问注册该节点下的镜像。"""
3
4         self._print_header("🕒 初始化向导：连接集群并注册资源")
5         self._print_step("连接 Master Server (PVE 集群中的任意可控节点)")
6
7         master_ip = self._prompt("Master Server IP/Hostname")
8
9         # 先问 PVE, 再问 SSH (避免顺序反直觉)
10        pve_user = self._prompt("PVE API 用户名", default="root@pam")
11        while True:
12            pve_pass = getpass("PVE API 密码 (必填)：")
13            if (pve_pass or "").strip():
14                break

```

```
15         self._print_error("PVE API 密码不能为空")
16
17     pve_api_port = 8006
18     pve_verify_ssl = False
19     if self._prompt_bool("是否配置 PVE API 高级参数 (端口/verify_ssl) ",
default=False):
20         pve_api_port = self._prompt_int("PVE API 端口", default=8006)
21         pve_verify_ssl = self._prompt_bool("PVE API verify_ssl", default=False)
22
23     ssh_user = self._prompt("SSH 用户名", default="root")
24     ssh_port = self._prompt_int("SSH 端口", default=22)
25     while True:
26         ssh_pass = getpass("SSH 密码 (必填; 集群内假设相同) : ")
27         if (ssh_pass or "").strip():
28             break
29         self._print_error("SSH 密码不能为空")
30     self.ssh_auth = SshAuth(username=ssh_user, password=ssh_pass,
port=ssh_port)
31
32     self._print_step("连接 Master (SSH + PVE API) ")
33     try:
34         with console.status("正在连接 Master (SSH + PVE API) ..."):
35             master = ServerHost(
36                 host_id="master",
37                 ip=master_ip,
38                 hostname="master",
39                 pve_node_name="master",
40                 ssh_username=ssh_user,
41                 ssh_password=ssh_pass,
42                 ssh_port=ssh_port,
43                 pve_api_username=pve_user,
44                 pve_api_password=pve_pass,
45                 pve_api_port=pve_api_port,
46                 pve_verify_ssl=pve_verify_ssl,
47             )
48     except Exception as e:
49         raise RuntimeError(f"连接 Master 失败: {e}") from e
50
51     # 尽量用 hostname 作为 pve_node_name
52     try:
53         out, _ = master.execute_ssh("hostname")
54         node_name = (out or "").strip().splitlines()[0].strip() if out else
"master"
55     except Exception:
56         node_name = "master"
57
58     master.hostname = node_name
```



```

59 master.pve_node_name = node_name
60 self.datacenter.master_server = master
61 self._dc_add_host(master, kind="Server", is_free=False)
62 self._print_success(f"Master 已连接: {master.hostname} ({master_ip})")
63
64 try:
65     clonezilla_isos = self._scan_clonezilla_isos(master, node_name)
66 except Exception as e:
67     self._print_error(f"[Clonezilla] 扫描 master 节点镜像失败: {e}")
68     clonezilla_isos = []
69
70 if clonezilla_isos:
71     self._clonezilla_seen_isos.update(clonezilla_isos)
72     self._register_clonezilla_isos(node_name, clonezilla_isos)
73
74 # 拉取节点列表
75 self._print_step("扫描集群节点 (pvsh get /nodes) ")
76 with console.status("正在扫描集群节点..."):
77     nodes = self._pvsh_get_json(master, "/nodes")
78
79 if not isinstance(nodes, list) or not nodes:
80     raise RuntimeError(f"未发现任何节点, pvsh 输出异常: {out}")
81
82 self._print_header("🚀 节点发现结果")
83 if RICH_AVAILABLE and _RichTable is not None and _rich_box is not None:
84     t = _RichTable(box=_rich_box.SIMPLE, show_lines=False)
85     t.add_column("#", justify="right", style="dim")
86     t.add_column("NODE", style="bold")
87     t.add_column("STATUS")
88     for idx, item in enumerate(nodes, start=1):
89         if not isinstance(item, dict):
90             continue
91         n = str(item.get("node") or "").strip() or "-"
92         status = str(item.get("status") or "").strip() or "-"
93         st_lower = status.lower()
94         st_style = (
95             "bold green"
96             if st_lower in {"online", "ok"}
97             else "bold red"
98             if st_lower in {"offline", "down"}
99             else "yellow"
100         )
101         t.add_row(str(idx), n, f"[{st_style}]{status}[/{st_style}]")
102     console.print(t)
103 else:
104     print(f"{'#':<4} {'NODE':<24} {'STATUS':<10}")
105     print("-" * 42)

```

```

1106         for idx, item in enumerate(nodes, start=1):
1107             if not isinstance(item, dict):
1108                 continue
1109             n = str(item.get("node") or "").strip() or "-"
1110             status = str(item.get("status") or "").strip() or "-"
1111             print(f"{idx:<4} {n:<24} {status:<10}")
1112
1113     self._print_header("🚀 交互式注册")
1114     vm_cache: dict[str, list[dict[str, Any]]] = {}
1115
1116     def _register_vms_for_node(node_name: str, node_vms: list[dict[str, Any]])
-> None:
1117         if not node_vms:
1118             self._print_info(f"[Image] 未发现节点 {node_name} 下的虚拟机")
1119             return
1120
1121         self._print_header(f"🧱 镜像候选 (节点 {node_name}) ")
1122
1123         if RICH_AVAILABLE and _RichTable is not None and _rich_box is not None:
1124             t = _RichTable(box=_rich_box.SIMPLE, show_lines=False)
1125             t.add_column("Status", no_wrap=True)
1126             t.add_column("VMID", no_wrap=True)
1127             t.add_column("Name")
1128             t.add_column("Node", no_wrap=True)
1129
1130             existing = set(getattr(self.dc.scheduler, "images", {}) or {})
1131             for vm in node_vms:
1132                 vmid = str(vm.get("vmid") or "").strip()
1133                 if not vmid:
1134                     continue
1135                 name = str(vm.get("name") or "").strip() or f"vm-{vmid}"
1136                 if str(vmid) in existing:
1137                     status = "[bold green]✅ registered[/bold green]"
1138                 else:
1139                     status = "[dim]➕ new[/dim]"
1140                 t.add_row(
1141                     status,
1142                     _escape_console_text(vmid),
1143                     _escape_console_text(name),
1144                     _escape_console_text(str(node_name)),
1145                 )
1146             console.print(t)
1147         else:
1148             for vm in node_vms:
1149                 vmid = str(vm.get("vmid") or "").strip()
1150                 name = str(vm.get("name") or "").strip() or f"vm-{vmid}"
1151                 self._print_info(f"VM {vmid} ({name})")

```

```

152
153         if not self._input_bool(f"是否为节点 {node_name} 注册 VM 镜像?",
default=False):
154             return
155
156         for vm in node_vms:
157             vmid = str(vm.get("vmid") or "").strip()
158             if not vmid:
159                 continue
160             name = str(vm.get("name") or "").strip() or f"vm-{vmid}"
161
162             if not self._input_bool(f"是否将 VM {vmid} ({name}) 注册为可调度镜
像?", default=False):
163                 continue
164
165             tags = self._input_csv_set('镜像标签 Tags (如 "navigation, slam")',
default="")
166             min_ram = self._input_int("最低硬件需求 Min RAM (GB)", default=4,
min_value=1)
167             need_gpu = self._input_bool("Need GPU?", default=False)
168
169             hw_req: dict[str, Any] = {"ram_gb": int(min_ram)}
170             if need_gpu:
171                 hw_req["has_gpu"] = True
172
173             image_id = str(vmid)
174             if image_id in getattr(self.dc.scheduler, "images", {}):
175                 self._print_info(f"[Image] 已存在镜像 {image_id}, 跳过")
176                 continue
177
178             img = RobotAppImage(
179                 image_id=image_id,
180                 name=name,
181                 version="pve",
182                 source_vm_id=int(vmid),
183                 is_template=bool(vm.get("template")),
184             )
185             # 记录镜像来源节点 (供 info image/host 查询展示; 不影响调度逻辑)
186             try:
187                 setattr(img, "source_node", str(node_name))
188             except Exception:
189                 pass
190             self.dc.register_image(
191                 img,
192                 image_profile={
193                     "labels": sorted(tags),
194                     "software": {},

```



```

195         "hardware": hw_req,
196     },
197     is_free=True,
198 )
199     self._print_success(f"[Image] 已注册镜像: {image_id} labels=
{sorted(tags)}")
200
201     # 在注册其它节点前, 先询问是否要注册 master 节点上的 VM
202     master_node = str(master.pve_node_name or "master")
203     if self._input_bool(f"是否注册 master 节点 {master_node} 的 VM?",
default=False):
204         try:
205             vm_cache[master_node] = self._list_node_vms(master, master_node)
206         except Exception as e:
207             self._print_error(f"[Image] 获取 master 节点 {master_node} 的 VM 列表
失败: {e}")
208             vm_cache[master_node] = []
209             _register_vms_for_node(master_node, vm_cache[master_node])
210
211     for item in nodes:
212         if not isinstance(item, dict):
213             continue
214
215         node = str(item.get("node") or "").strip()
216         status = str(item.get("status") or "").strip()
217         if not node:
218             continue
219
220         if node == master.pve_node_name:
221             continue
222
223         self._print_header(f"🚀 节点注册: {node} (status={status})")
224         role = self._prompt("该节点是 [S]erver 还是 [R]obot? (输入 i 跳过)",
default="i").strip().lower()
225         if role in {"i", "skip"}:
226             self._print_info("[Node] 已跳过")
227             continue
228
229         # 自动解析节点 IP: 只在无法解析时才回退询问
230         addr = self._resolve_node_ssh_ip(master, node)
231         if not addr:
232             addr = self._prompt("无法自动解析节点 IP, 请手动输入该节点可 SSH 的
IP/Hostname", default="").strip()
233         if not addr:
234             self._print_error("[Node] 未提供可 SSH 地址, 已跳过")
235             continue
236         self._print_info(f"[Node] SSH 地址: {addr}")

```

```

237
238     kind = "Server"
239     is_free = False
240     labels: set[str] = set()
241     capabilities: dict[str, Any] = {}
242
243     try:
244         if role in {"s", "server"}:
245             kind = "Server"
246             is_free = False
247             host = ServerHost(
248                 host_id=node,
249                 ip=addr,
250                 hostname=node,
251                 pve_node_name=node,
252                 connect_pve_api=False,
253                 ssh_username=ssh_user,
254                 ssh_password=ssh_pass,
255                 ssh_port=ssh_port,
256             )
257             self._dc_add_host(host, kind=kind, is_free=is_free)
258         elif role in {"r", "robot"}:
259             kind = "PVE"
260             is_free = True
261             cpu_cores = self._input_int("cpu_cores", default=4,
min_value=1)
262             ram_gb = self._input_int("ram_gb", default=4, min_value=1)
263             has_gpu = self._input_bool("has_gpu", default=False)
264             labels = self._input_csv_set('labels (逗号分隔, 如 "lab,
ground_floor")', default="")
265             capabilities = {"cpu_cores": cpu_cores, "ram_gb": ram_gb,
"has_gpu": has_gpu}
266
267             # 关键: 流水线在 NetworkPrep 阶段要求目标端 PveHost.network_config
非空。
268
269             # 这里用 SSH 自动探测/环境变量兜底生成 network_config。
270             try:
271                 self._print_step(f"自动生成 {node} 的 network_config")
272                 node_net_cfg = self._auto_network_config(ip=addr,
node_name=node)
273             except Exception as e:
274                 raise RuntimeError(
f"无法为节点 {node} 生成 network_config (将导致流水线失
败) 。"
f"请检查 SSH 可达性或设置环境变量
PVE_GATEWAY/PVE_PREFIXLEN 等。原因: {e}"
) from e

```

```

277
278         host = PveHost(
279             host_id=node,
280             ip=addr,
281             hostname=node,
282             pve_node_name=node,
283             network_config=None,
284             connect_pve_api=False,
285             ssh_username=ssh_user,
286             ssh_password=ssh_pass,
287             ssh_port=ssh_port,
288         )
289         host.network_config = dict(node_net_cfg or {})
290         self._dc_add_host(
291             host,
292             kind=kind,
293             is_free=is_free,
294             labels=labels,
295             capabilities=capabilities,
296         )
297     else:
298         self._print_error("[Node] 输入无效, 仅支持 S/R/i, 已跳过")
299         continue
300 except Exception as e:
301     self._print_error(f"[Node] 注册失败: {e}")
302     continue
303
304 # 镜像注册: 在节点注册后立即询问该节点下的 VM
305 if node not in vm_cache:
306     try:
307         vm_cache[node] = self._list_node_vms(master, node)
308     except Exception as e:
309         self._print_error(f"[Image] 获取节点 {node} 的 VM 列表失败: {e}")
310         vm_cache[node] = []
311
312 node_vms = vm_cache[node]
313
314 _register_vms_for_node(node, node_vms)
315
316 self._print_success("初始化完成")
317 self._print_info("输入 help 查看可用指令; 输入 list 查看注册资源")

```

镜像定期更新

机器人镜像并不是静止不动的, 在实际应用中, 它可能会迁移到不同的机器人主机上, 也可能被克隆出新的实例。所以系统不能只在镜像创建时知道它在哪, 还要定期刷新它当前的位置。在系统中, 数

据中心模块会周期性读取 PVE 资源信息，更新每个镜像当前所在的节点。可以把这个过程理解成“**系统定期盘点库存**”。这样，后续无论是调度还是部署，都能基于最新状态进行决策。具体实现代码如下：

代码块

```

1 def _cmd_refresh(self) -> None:
2     master = self.datacenter.master_server
3     if master is None:
4         self._print_error("[refresh] master_server 未设置")
5         return
6
7     with console.status("正在刷新资源 (nodes/vms/status) ..."):
8         # 尝试确保 API/SSH 可用；失败则尝试重连
9         try:
10             if getattr(master, "pve_api", None) is not None:
11                 _ = master.pve_api.version.get()
12         except Exception:
13             try:
14                 new_master = ServerHost(
15                     host_id=str(getattr(master, "host_id", "master")),
16                     ip=str(getattr(master, "ip", "")),
17                     hostname=str(getattr(master, "hostname", "master")),
18                     pve_node_name=str(getattr(master, "pve_node_name",
19 "master"))),
20                     ssh_username=str(getattr(master, "_ssh_username", "root")),
21                     ssh_password=getattr(master, "_ssh_password", None),
22                     ssh_port=int(getattr(master, "_ssh_port", 22)),
23                     pve_api_username=str(getattr(master, "_pve_api_username",
24 "root@pam")),
25                     pve_api_password=getattr(master, "_pve_api_password",
None),
26                     pve_api_port=int(getattr(master, "_pve_api_port", 8006)),
27                     pve_verify_ssl=bool(getattr(master, "_pve_verify_ssl",
False))),
28                 )
29                 master = new_master
30                 self.datacenter.master_server = new_master
31
32                 # 同步到 datacenter/scheduler host 表
33                 try:
34                     self.datacenter.hosts[str(new_master.host_id)] = new_master
35                 except Exception:
36                     pass
37
38             try:

```

```
38         sch = getattr(self.datacenter, "scheduler", None)
39         if sch is not None:
40             sch.hosts[str(new_master.host_id)] = new_master
41     except Exception:
42         pass
43
44     except Exception as e:
45         self._print_error(f"[refresh] 重连 master 失败: {e}")
46         return
47
48     # 刷新节点在线状态
49     try:
50         nodes = self._pvesh_get_json(master, "/nodes")
51     except Exception as e:
52         self._print_error(f"[refresh] 获取 nodes 失败: {e}")
53         return
54
55     online = 0
56     total = 0
57
58     if isinstance(nodes, list):
59         total = len(nodes)
60
61         for item in nodes:
62             if not isinstance(item, dict):
63                 continue
64
65             node = str(item.get("node") or "").strip()
66             status = str(item.get("status") or "").strip() or "unknown"
67
68             if status.lower() in {"online", "ok"}:
69                 online += 1
70
71             h = (self.datacenter.hosts or {}).get(node)
72             if h is not None:
73                 try:
74                     setattr(h, "node_status", status)
75                 except Exception:
76                     pass
77
78     # 刷新 VM/镜像位置 (仅更新已纳管镜像的位置信息)
79     try:
80         self.datacenter.refresh_image_locations()
81     except Exception:
82         pass
83
```

```
84         self._print_success(f"[refresh] 完成: nodes={online}/{total} online; 已刷新已  
纳管镜像位置")
```

镜像模板化与克隆

服务器端还会提供一个对机器人镜像的功能，就是模板化和克隆，如果一张机器人镜像已经被验证可用，那么我们就可以把它做成模板。**模板可以理解成一份标准镜像**。后续如果有新的部署需求，就不必重新从零搭建环境，而是直接从这个模板快速克隆出一个新的实例。

这样一来，部署工作就从“人工重建环境”，变成了“自动复制环境”，能够消除手动配置差异导致的运行风险，极大地缩短了新机器人投入使用的准备周期。具体实现代码如下：

代码块

```
1  #####
2      #1. 镜像模板化
3  #####
4  def _cmd_template(self, vmid_raw: str) -> None:
5      master = self.datacenter.master_server
6      if master is None:
7          self._print_error("[template] master_server 未设置")
8          return
9
10     scheduler = getattr(self.datacenter, "scheduler", None)
11     if scheduler is None:
12         self._print_error("[template] scheduler 未初始化")
13         return
14
15     try:
16         vmid = int(vmid_raw)
17     except ValueError:
18         self._print_error("[template] vmid 必须是整数")
19         return
20
21     image_id = str(vmid)
22     if image_id not in (getattr(scheduler, "images", {}) or {}):
23         self._print_error(f"[template] VM {vmid} 未在平台注册")
24         return
25
26     if image_id in set(getattr(scheduler, "busy_images", set()) or set()):
27         self._print_error(f"[template] VM {vmid} 当前处于 BUSY 状态，不能执行模板  
化")
28         return
29
30     try:
31         current_node = self._find_vm_current_node(master, vmid)
32     except Exception as e:
```



```
33         self._print_error(f"[template] 获取 VM {vmid} 所在节点失败: {e}")
34         return
35
36     if not current_node:
37         self._print_error(f"[template] 无法找到 VM {vmid} 所在节点")
38         return
39
40     controller = None
41     temporary_controller = False
42     manual_logger, log_path = self._setup_manual_logger("template")
43     self._print_step("任务已开始")
44     self._print_info(f"日志写入: {log_path}")
45
46     try:
47         controller, temporary_controller =
self._get_qm_controller_for_node(master, current_node)
48         already_template = bool(controller.is_template_vm(vmid))
49         if not already_template:
50             out, err = controller.template_vm(vmid)
51             if (out or "").strip():
52                 manual_logger.info("[template] stdout: %s", (out or
"").strip())
53             if (err or "").strip():
54                 manual_logger.info("[template] stderr: %s", (err or
"").strip())
55
56             self._set_registered_image_template_flag(image_id, True)
57             img = (getattr(scheduler, "images", {}) or {}).get(image_id)
58             if img is not None:
59                 setattr(img, "source_node", str(current_node))
60
61         if already_template:
62             self._print_success(f"[template] VM {vmid} 已经是模板")
63         else:
64             manual_logger.info("[template] success: vmid=%s node=%s", vmid,
current_node)
65             self._print_success(f"[template] VM {vmid} 已模板化")
66     except Exception as e:
67         manual_logger.exception("[template] failed")
68         self._print_error(f"[template] 失败原因: {e}")
69         self._print_error(f"[template] 模板化失败, 详见日志: {log_path}")
70         self._print_suggestion_for_exception(e)
71     finally:
72         if temporary_controller and controller is not None:
73             try:
74                 controller.stop_listening()
75             except Exception:
```

```

76         pass
77     def template_vm(self, image_id: int) -> tuple[str, str]:
78         image_id = int(image_id)
79         status = self.get_vm_runtime_status(image_id)
80         if status == "running":
81             raise RuntimeError(f"VM {image_id} 当前处于运行状态, 请先关机后再模板化")
82         if self.is_template_vm(image_id):
83             return "", ""
84
85         out, err, rc = self._execute_ssh_with_status(f"qm template {image_id}")
86         text = (out or "") + (err or "")
87         if rc != 0:
88             raise RuntimeError(text.strip() or f"模板化 VM {image_id} 失败")
89         return out, err
90     #####
91     #2. 镜像模板克隆
92     #####
93     def _cmd_clone_template(
94         self,
95         template_vmid_raw: str,
96         new_vmid_raw: str,
97         target_node_id: str,
98         new_name: str | None = None,
99     ) -> None:
100         master = self.datacenter.master_server
101         if master is None:
102             self._print_error("[clone-template] master_server 未设置")
103             return
104
105         scheduler = getattr(self.datacenter, "scheduler", None)
106         if scheduler is None:
107             self._print_error("[clone-template] scheduler 未初始化")
108             return
109
110         try:
111             template_vmid = int(template_vmid_raw)
112             new_vmid = int(new_vmid_raw)
113         except ValueError:
114             self._print_error("[clone-template] template_vmid 和 new_vmid 都必须都是整
115 数")
116             return
117
118         source_image_id = str(template_vmid)
119         new_image_id = str(new_vmid)
120         if source_image_id not in (getattr(scheduler, "images", {}) or {}):
121             self._print_error(f"[clone-template] 模板源 VM {template_vmid} 未在平台注
册")

```

```
121         return
122     if new_image_id in (getattr(scheduler, "images", {}) or {}):
123         self._print_error(f"[clone-template] 新 VMID {new_vmid} 已在平台注册")
124         return
125
126     try:
127         existing_node = self._find_vm_current_node(master, new_vmid)
128     except Exception as e:
129         self._print_error(f"[clone-template] 检查目标 VMID 是否已存在失败: {e}")
130         return
131     if existing_node:
132         self._print_error(f"[clone-template] VMID {new_vmid} 已存在于节点
133 {existing_node}")
134         return
135
136     try:
137         current_node = self._find_vm_current_node(master, template_vmid)
138     except Exception as e:
139         self._print_error(f"[clone-template] 获取模板所在节点失败: {e}")
140         return
141     if not current_node:
142         self._print_error(f"[clone-template] 无法找到模板 VM {template_vmid} 所在
143 节点")
144         return
145
146     try:
147         target_node = self._resolve_target_pve_node(target_node_id)
148     except Exception as e:
149         self._print_error(f"[clone-template] 目标节点无效: {e}")
150         return
151
152     source_image = (getattr(scheduler, "images", {}) or
153 {}).get(source_image_id)
154     source_name = str(getattr(source_image, "name", "") or f"vm-
155 {template_vmid}").strip()
156     clone_name = str(new_name or "").strip() or f"{source_name}-clone-
157 {new_vmid}"
158
159     controller = None
160     temporary_controller = False
161     manual_logger, log_path = self._setup_manual_logger("clone-template")
162     self._print_step("任务已开始")
163     self._print_info(f"日志写入: {log_path}")
164
165     try:
166         controller, temporary_controller =
167 self._get_qm_controller_for_node(master, current_node)
```

```

162         if not controller.is_template_vm(template_vmid):
163             raise RuntimeError(f"VM {template_vmid} 不是模板, 不能执行模板克隆")
164         self._set_registered_image_template_flag(source_image_id, True)
165
166         out, err = controller.clone_template_vm(
167             template_vmid,
168             new_vmid,
169             target_node=target_node,
170             name=clone_name,
171             full=True,
172         )
173         if (out or "").strip():
174             manual_logger.info("[clone-template] stdout: %s", (out or
175             "").strip())
176         if (err or "").strip():
177             manual_logger.info("[clone-template] stderr: %s", (err or
178             "").strip())
179
180         self._register_cloned_image(source_image_id, new_vmid, clone_name,
181         target_node)
182         manual_logger.info(
183             "[clone-template] success: template_vmid=%s new_vmid=%s target=%s
184             name=%s",
185             template_vmid,
186             new_vmid,
187             target_node,
188             clone_name,
189         )
190         self._print_success(
191             f"[clone-template] 已从模板 {template_vmid} 克隆 VM {new_vmid} 到节
192             点 {target_node}"
193         )
194     except Exception as e:
195         manual_logger.exception("[clone-template] failed")
196         self._print_error(f"[clone-template] 失败原因: {e}")
197         self._print_error(f"[clone-template] 克隆失败, 详见日志: {log_path}")
198         self._print_suggestion_for_exception(e)
199     finally:
200         if temporary_controller and controller is not None:
201             try:
202                 controller.stop_listening()
203             except Exception:
204                 pass
205
206 def clone_template_vm(
207     self,
208     template_vmid: int,
209     new_vmid: int,

```

```

204     *,
205     target_node: str,
206     name: str | None = None,
207     full: bool = True,
208 ) -> tuple[str, str]:
209     template_vmid = int(template_vmid)
210     new_vmid = int(new_vmid)
211     if not self.is_template_vm(template_vmid):
212         raise RuntimeError(f"VM {template_vmid} 不是模板，无法执行模板克隆")
213
214     args: list[str] = ["qm", "clone", str(template_vmid), str(new_vmid)]
215     if full:
216         args.extend(["--full", "1"])
217     if target_node:
218         args.extend(["--target", shlex.quote(str(target_node))])
219     if name:
220         args.extend(["--name", shlex.quote(str(name))])
221
222     out, err, rc = self._execute_ssh_with_status(" ".join(args))
223     text = (out or "") + (err or "")
224     if rc != 0:
225         raise RuntimeError(text.strip() or f"从模板 {template_vmid} 克隆 VM
{new_vmid} 失败")
226     return out, err

```

到这里，我们解决了如何把机器人镜像进行统一表示、统一存储和统一管理。接下来，自然就会进入两个更进一步的问题：第一，如何把部署流程 pipeline 化，实现自动部署；第二，面对不同硬件条件和任务约束，如何实现智能分发和调度。这些内容会在后续的视频中继续展开，敬请期待。

实现效果

演示：注册服务器，定时更新镜像状态

